L06 — Searching: Linear Search and Binary Search with Complexity Analysis

Unit: 1 | **Chapter:** Fundamentals of Data Structures & Arrays | **BT Level:** BT4 | **CO:** CO1, CO5

Learning Objectives

- Implement linear search and analyze its complexity
- Implement binary search (iterative and recursive) and analyze its complexity
- State the precondition for binary search
- Compare both algorithms and choose appropriately

---

1. Introduction to Searching

Searching is the process of finding the location of a target element within a data structure.

Two fundamental search algorithms on arrays:

1. **Linear Search** — works on any array (sorted or unsorted)
2. **Binary Search** — works only on **sorted** arrays, dramatically faster

---

2. Linear Search

2.1 Algorithm

Scan each element one by one from left to right until the target is found or the array is exhausted.

```
def linear_search(arr, n, target):
    for i in range(n):
        if arr[i] == target:
            return i        # Found at index i
    return -1               # Not found
```

2.2 Trace Example

Array: `[45, 12, 78, 3, 56, 21]`, Target: `56`

Step	i	arr[i]	Match?
1	0	45	No
2	1	12	No
3	2	78	No
4	3	3	No

5 4 56 **Yes → return 4**

2.3 Complexity Analysis

Case	Condition	Comparisons	Time
Best	Target at index 0	1	O(1)
Worst	Target at last index or absent	n	O(n)
Average	Target at random position	n/2	O(n)

Space: O(1) — no extra memory

2.4 Improved Linear Search: Sentinel Search

Avoids checking the loop boundary on every iteration:

```
def sentinel_search(arr, n, target):
    last = arr[n - 1]
    arr[n - 1] = target    # Place sentinel at end
    i = 0
    while arr[i] != target:
        i += 1
    arr[n - 1] = last      # Restore
    if i < n - 1 or arr[n - 1] == target:
        return i
    return -1
```

Reduces the number of comparisons per iteration from 2 to 1.

---

3. Binary Search

3.1 Precondition

> **The array MUST be sorted** (ascending or descending).

Binary search exploits the sorted order to **eliminate half the search space** in each step.

3.2 Core Idea

Maintain three pointers: `low`, `mid`, `high`.

- If `arr[mid] == target`: found!
- If `arr[mid] < target`: target must be in right half → `low = mid + 1`
- If `arr[mid] > target`: target must be in left half → `high = mid - 1`

3.3 Iterative Binary Search

```
def binary_search_iterative(arr, n, target):
    low = 0
    high = n - 1
```

```
    while low <= high:
        mid = low + (high - low) // 2   # Avoids integer overflow vs (l

        if arr[mid] == target:
            return mid          # Found
        elif arr[mid] < target:
            low = mid + 1       # Search right half
        else:
            high = mid - 1      # Search left half

    return -1                   # Not found
```

Note: Use `mid = low + (high - low) // 2` instead of `(low + high) // 2` to prevent integer overflow in languages with fixed-width integers.

3.4 Recursive Binary Search

```
def binary_search_recursive(arr, low, high, target):
    if low > high:
        return -1              # Base case: not found

    mid = low + (high - low) // 2

    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search_recursive(arr, mid + 1, high, target)
    else:
        return binary_search_recursive(arr, low, mid - 1, target)
```

3.5 Trace Example

Array: `[3, 12, 21, 45, 56, 78]`, Target: `45`, n = 6

Iteration	low	high	mid	arr[mid]	Decision
1	0	5	2	21	21 < 45 → right half
2	3	5	4	56	56 > 45 → left half
3	3	3	3	45	**Found! Return 3**

Only 3 comparisons to find 45 in 6 elements. Linear search would take 4.

3.6 Complexity Analysis

Each iteration halves the search space:

After k iterations, search space = n / 2^k

Algorithm terminates when search space = 1: $$n / 2^k = 1 \implies k = \log_2(n)$$

Case	Comparisons	Time
Best	1	O(1)

Worst	$\lfloor \log_2(n) \rfloor + 1$	O(log n)
Average	O(log n)	O(log n)

Space:

- Iterative: O(1)
- Recursive: O(log n) — call stack depth

---

4. Comparison: Linear vs Binary Search

Property	Linear Search	Binary Search
Precondition	None	Array must be sorted
Best case	O(1)	O(1)
Worst case	O(n)	O(log n)
Average case	O(n)	O(log n)
Space (iterative)	O(1)	O(1)
Works on linked list	Yes	No (no random access)
Implementation	Simple	Moderate

When to Use Each

Situation	Choose
Unsorted data	Linear Search
Sorted data, frequent queries	Binary Search
Small array ($n < 50$)	Either (difference negligible)
Single search in sorted data	Linear (avoids sort overhead)

---

5. Performance at Scale

For $n = 1{,}000{,}000$:

- Linear search: up to **1,000,000** comparisons
- Binary search: at most **20** comparisons ($\log_2(10^6) \approx 20$)

---

6. Variants of Binary Search

Find First Occurrence

```
def first_occurrence(arr, n, target):
    low, high, result = 0, n - 1, -1
    while low <= high:
        mid = low + (high - low) // 2
        if arr[mid] == target:
            result = mid
```

```
            high = mid - 1   # Continue searching left
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return result
```

Find Last Occurrence

```
def last_occurrence(arr, n, target):
    low, high, result = 0, n - 1, -1
    while low <= high:
        mid = low + (high - low) // 2
        if arr[mid] == target:
            result = mid
            low = mid + 1    # Continue searching right
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return result
```

Summary

- **Linear search:** Simple, O(n) worst case, works on any array.
- **Binary search:** Requires sorted array, O(log n) worst case — drastically faster for large n.
- Binary search eliminates half the search space per step by leveraging sorted order.
- For $n = 10^6$, binary search takes $\sim$20 steps vs up to 10^6 for linear search.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 2.1 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 4.9 (Springer, 2020)
- Laaksonen, *Guide to Competitive Programming*, Ch. 3 (Springer, 2024)